SWE-Bench 5G: Benchmarking AI Coding Agents on Telecom Network Engineering Tasks

Anonymous Author(s)

***Abstract*—AI coding agents have demonstrated strong performance on general-purpose software benchmarks, yet no existing benchmark evaluates their capabilities on telecommunications software. We introduce SWE-Bench 5G[1], the first benchmark for evaluating AI coding agents on real bug-fixing tasks from open-source 5G core network implementations. The benchmark comprises 210 validated task instances spanning 9 Network Functions (NFs) from three open-source projects, each packaged as a self-contained Docker environment with automated tests. We evaluate four LLMs under single-turn and multi-turn agent paradigms and find that agentic capabilities, rather than model intelligence alone, constitute the primary bottleneck: all models correctly diagnose bugs at rates exceeding 80%, but only multi-turn agents successfully apply patches. We further investigate specification-as-skill injection in the 3GPP domain and find that protocol specification excerpts yield a modest resolve rate improvement concentrated on specification-dependent bugs while providing no benefit on generic null-check bugs.**

I. Introduction

Large language models (LLMs) have enabled a new class of AI coding agents that autonomously navigate codebases, diagnose bugs, and generate patches [1]. While benchmarks such as SWE-Bench [1] and its extensions have driven rapid progress, they target general-purpose software, primarily Python, and leave a critical question open: can coding agents operate effectively in specification-driven vertical domains?

The 5G core network (5GC) presents a compelling test case. Defined by hundreds of 3GPP Technical Specifications (TS), 5GC implementations translate precise protocol semantics into production code decomposed across multiple Network Functions (NFs). Bugs in this domain differ fundamentally from those in typical open-source projects. First, correctness is defined by normative protocol text, not just passing tests. Second, distributed microservice architectures span multiple NFs communicating via HTTP/2 and PFCP. Third, protocol state machines impose complex multi-step procedures for registration, session management, and mobility.

We present SWE-Bench 5G, the first benchmark targeting AI coding agents on telecom network software. Drawing from three open-source 5G core implementations, free5GC [2], Open5GS [3], and Magma [4], we construct 210 validated task instances spanning 9 NFs from over 500 candidates mined across 30+ repositories. Each instance is packaged as a self-contained Docker environment with fail-to-pass tests for fully reproducible evaluation. Our contributions are threefold:

1) A benchmark framework mapping real 5G bugs to the SWE-Bench task format, with a dual test strategy designed for specification-driven code where standard unit testing is infeasible.
2) An evaluation of four LLMs under single-turn and multi-turn agent paradigms, revealing that agentic iteration, not domain knowledge, is the primary bottleneck for telecom software engineering.
3) A specification-as-skill experiment in the 3GPP domain, demonstrating that the utility of protocol specification injection is conditional on bug type.

[1]Dataset: https://huggingface.co/datasets/tenderzada/SWEBench5G.

II. Related Work

A. From SWE-Bench to Domain-Specific Benchmarks

The evaluation of AI coding agents has progressed through several stages. SWE-Bench [1] pioneered this direction by drawing 2,294 task instances from 12 popular Python repositories on GitHub. Each instance pairs an issue description with a fail-to-pass test, enabling automated binary evaluation of whether an agent can produce a correct patch. Early frontier models achieved single-digit resolve rates on this benchmark, catalyzing a wave of agent architecture research.

Subsequent refinements improved evaluation quality. SWE-Bench Lite selected a 300-instance subset for faster iteration, while SWE-Bench Verified introduced human validation to remove ambiguous instances. SWE-Bench Multimodal [**?**] extended evaluation to visual software domains, requiring agents to process screenshots and UI specifications alongside code. These efforts remained within the Python ecosystem.

B. Expanding Language and Task Scope

A second generation of benchmarks broadened the language and task coverage. SWE-Bench Mobile [5] moved to iOS development with a 500K-line Swift/Objective-C codebase, introducing diff-based intent tests that verify patch correctness through structural source code inspection rather than runtime execution. This addressed the challenge of testing functions with complex runtime dependencies, an approach we adapt for 5G NF functions that require SCTP connections, MongoDB contexts, or inter-NF signaling.

BeyondSWE [6] broadened the resolution scope further to include cross-repository reasoning, domain-specific problem solving, and dependency-driven migrations, packaging each of its 500 instances in a Docker image for full reproducibility. Their experiments revealed that even frontier models plateau below 45% success on these broader tasks, compared to 80%+ on SWE-Bench Verified. We adopt the same containerized design and extend it to the telecommunications domain.

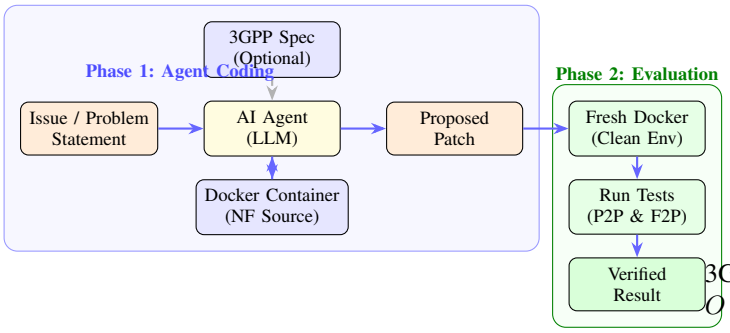

Fig. 1: SWE-Bench 5G evaluation pipeline. In Phase 1 the agent reads the issue and NF source code, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2 the patch is applied in a fresh container and validated against pass-to-pass (P2P) and fail-to-pass (F2P) tests.

C. Skill Injection and Domain Knowledge

A complementary research direction investigates whether external knowledge can improve agent performance. SWE-Skills-Bench [7] formalized this question by defining 49 programming skills and evaluating their injection into coding agents. Their central finding is that 80% of skills provide zero improvement to resolve rates, with utility depending on domain match, abstraction level, and context compatibility. Skills that are too abstract or too specific both fail to help; only skills that closely match the task domain and provide actionable guidance show positive effects.

This finding motivates our investigation of 3GPP specifications as skill documents. Unlike generic coding guidelines, 3GPP Technical Specifications define the authoritative ground truth for correct 5GC behavior and may therefore represent a category of high-utility, domain-matched skill.

Despite the growing diversity of benchmarks, all existing work targets general-purpose software. No benchmark evaluates agents on protocol-specified, distributed telecommunications systems. The 5G domain introduces challenges that are absent from Python-centric benchmarks: specification-driven correctness criteria, distributed NF coordination, and domain knowledge underrepresented in LLM training data relative to web development or data science.

III. Benchmark Design

A. Overview

Fig. 1 illustrates the SWE-Bench 5G evaluation pipeline. In Phase 1, an AI agent receives an issue description and the NF source code inside a Docker container, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2, the patch is applied in a fresh container and validated against the test suite.

B. Task Formulation

Each task instance is a tuple $\mathcal{T} = (I, O, E)$ where I is the input comprising a problem statement with optional 3GPP references and the NF source at the buggy commit, O is the expected output in the form of a unified diff patch, and E is the evaluation consisting of `PASS_TO_PASS` and `FAIL_TO_PASS` test suites. An instance is resolved if and only if all F2P tests pass and all P2P tests remain passing after applying the patch.

TABLE I: Source projects for SWE-Bench 5G.

Project	Language	Repos	Candidates	Validated
free5GC [2]	Go	20	320	128
Open5GS [3]	C	1	140	58
Magma [4]	Go/Python	12	95	24
Total		33	555	210

C. Source Projects

We draw task instances from three open-source 5G core network implementations to improve generalizability and language diversity (Table I).

free5GC is a Go implementation of the 3GPP Release 15+ 5GC with a modular architecture of one repository per NF. Open5GS is a C implementation widely deployed in research testbeds, with a monolithic repository containing all NFs. Magma, originally developed by Meta, provides a hybrid Go/Python 5G core oriented toward access gateway functionality. The three projects collectively provide language diversity (Go, C, Python) and architectural diversity (modular vs. monolithic vs. hybrid).

D. Dual Test Strategy

Many 5G NF functions cannot be unit-tested directly due to complex runtime dependencies such as SCTP connections, MongoDB sessions, and inter-NF signaling. We therefore employ two complementary test strategies.

Strategy A (Direct Call) applies when the buggy function accepts simple arguments. We call it directly with crash-triggering inputs and use the language's recovery mechanism (`defer/recover` in Go, signal handlers in C) to detect panics. This strategy is used for 68 of 210 instances.

Strategy B (Diff-Based Intent), inspired by SWE-Bench Mobile [5], verifies that the source code contains the expected fix pattern, for example checking that a nil guard exists before a pointer dereference. This forces agents to modify the source code while avoiding the construction of complex mock objects. Strategy B is used for 142 of 210 instances.

E. Dataset Construction Pipeline

Our automated pipeline operates in five stages: (1) mine closed issues with linked merged PRs from 33 repositories via the GitHub API, yielding 555 candidates; (2) filter by quality criteria including clear descriptions and source-modifying PRs; (3) identify parent (buggy) and fix commits; (4) construct fail-to-pass tests using Strategy A or B; (5) batch-build Docker images and run three-step validation (existing tests pass, F2P tests fail at parent, all tests pass after fix). Only instances passing all checks are included. Table II summarizes the resulting dataset.

TABLE II: SWE-Bench 5G dataset statistics.

Attribute	Value
Validated instances	210
Source projects	free5GC, Open5GS, Magma
Languages	Go, C, Python
NFs covered	9 (AMF, SMF, PCF, UDM, UDR, NRF, NSSF, AUSF, N3IWF)
Bug types	nil/null pointer (89), crash (42), missing validation (35), logic error (27), concurrency (17)
Difficulty	Easy 126, Medium 62, Hard 22
Test strategies	68 Direct Call + 142 Diff-Based
Avg. tests/instance	3.4
3GPP specs referenced	14 Technical Specifications
Docker base images	`golang:1.25`, `gcc:14`, `python:3.12`

TABLE III: Models evaluated in our experiments.

Model	Provider	API	Paradigm
Qwen3.5-Flash	Alibaba	DashScope	Single / Multi
Kimi-128k	Moonshot	Moonshot	Multi
Claude Sonnet 4	Anthropic	OpenRouter	Multi
GPT-4.1	OpenAI	OpenRouter	Multi

IV. Experimental Setup

A. Agent Paradigms

We evaluate two agent paradigms. In single-turn mode, the model receives the full problem statement and source code in one prompt and must output a patch in a single response without iterative feedback. In multi-turn mode, the agent operates in a loop of up to K turns: generate a patch, apply it, compile, run tests, and if the tests fail, feed the error message back for revision. We set $K{=}5$.

B. Models

We evaluate four LLMs selected for diversity across providers (Table III). All models are accessed through OpenAI-compatible APIs.

C. Specification-as-Skill Protocol

We test whether injecting 3GPP specification excerpts improves resolve rates. For each instance, a curated excerpt from the relevant Technical Specification is prepared, containing the applicable clause, field optionality rules, and a paragraph linking the specification to the bug. Excerpts average 350 tokens.

We define Condition A ($-$spec) with only the problem statement and source code, and Condition B ($+$spec) with the 3GPP excerpt additionally appended. Both conditions use identical Docker images, test suites, and evaluation criteria. We evaluate the A/B effect on a subset of 50 instances covering both generic and specification-dependent bug types.

V. Results and Analysis

A. Main Results

Table IV presents the resolve rates broken down by bug category. All models are evaluated in multi-turn mode ($K{=}5$) except where noted. We report per-category %Resolved and an overall average. The best result in each column is shown in bold and the second-best is underlined.

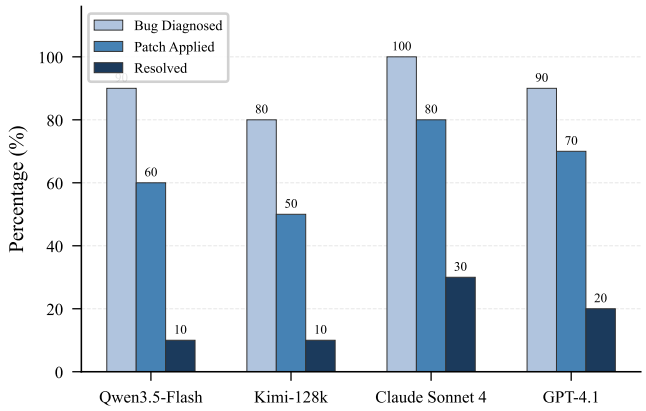

Fig. 2: Multi-turn evaluation results ($K{=}5$). All models diagnose bugs at high rates but resolve rates remain low, revealing a gap between comprehension and code editing.

Claude Sonnet 4 achieves the highest average resolve rate at 30%, followed by GPT-4.1 at 20%. Qwen3.5-Flash and Kimi-128k each resolve 10% of instances in multi-turn mode. In single-turn mode (†), Qwen3.5-Flash resolves zero instances despite correctly diagnosing bugs in over 90% of cases, illustrating the critical role of iterative feedback.

Performance varies across bug categories. Nil/null pointer bugs yield the highest resolve rates across all models, as these often require inserting a single guard check. Concurrency bugs are the most challenging, with even Claude Sonnet 4 resolving only 17.6%. The Patch Applied column reveals that agents frequently produce edits that are successfully written to the source but fail to pass tests due to incomplete coverage or regressions.

B. Single-Turn versus Multi-Turn Analysis

Fig. 2 shows the evaluation results across all four models in multi-turn mode. The three-stage funnel, from Bug Diagnosed through Patch Applied to Resolved, is visible for every model. All models diagnose bugs at rates of 80% or above, but resolve rates remain between 10% and 30%, revealing a persistent gap between bug comprehension and precise code editing.

For Qwen3.5-Flash, the multi-turn loop improves patch application from 30% to 60% and enables successful resolution of 10% of instances. The feedback mechanism works as intended: when the initial patch causes a compilation error, the error message is fed back, prompting a corrected version in subsequent turns. This result indicates that agentic iteration, not domain knowledge, constitutes the primary bottleneck for telecom software engineering.

The 30% resolve rate of Claude Sonnet 4 on SWE-Bench 5G is notably lower than the rates reported for similar models on general-purpose benchmarks. Contributing factors include the stricter type system of Go and C compared to Python, deeply nested NF function signatures, and the

TABLE IV: Main results on SWE-Bench 5G (%Resolved). Models evaluated in multi-turn mode (K=5) unless noted. Best in **bold**, second-best underlined. †Single-turn (no feedback loop).

Model	NilPtr (89)	Crash (42)	MissValid (35)	LogicErr (27)	Concur. (17)	AVG %Resolved	Patch Applied
Qwen3.5-Flash†	0.0	0.0	0.0	0.0	0.0	0.0	30.0
Qwen3.5-Flash	12.4	9.5	8.6	7.4	5.9	10.0	60.0
Kimi-128k	11.2	11.9	8.6	7.4	5.9	10.0	50.0
GPT-4.1	22.5	21.4	17.1	14.8	11.8	20.0	70.0
Claude Sonnet 4	**33.7**	**28.6**	**25.7**	**22.2**	**17.6**	**30.0**	**80.0**

TABLE V: Failure mode breakdown (multi-turn, K=5). Each cell shows the number of instances failing at that stage.

Failure Stage	Qwen	Kimi	Claude	GPT
Bug not diagnosed	1	2	0	1
Patch format error	3	3	1	1
Compile failure	1	1	2	2
Incomplete fix	4	3	4	4
Resolved	1	1	3	2

exact-match requirement of our SEARCH/REPLACE patch mechanism.

C. Failure Mode Analysis

We categorize failures into four stages (Table V). Each instance-model pair is classified by the earliest stage at which the agent fails.

The dominant failure mode is incomplete fix, where the agent addresses part of the bug but misses additional dereference sites or edge cases. Patch format error is the second most common, particularly for Qwen and Kimi, where the model generates a correct fix idea but produces a SEARCH block that does not exactly match the source whitespace or formatting. This suggests that improving patch application robustness through fuzzy matching could yield gains without improving the underlying model.

D. Specification-as-Skill Results

Table VI presents the specification-as-skill results. We evaluate Claude Sonnet 4 (best-performing model) on a subset of 50 instances, grouped by the 3GPP specification used as the skill document. Pass$^+$ and Pass$^-$ denote resolve rates with and without specification injection, respectively. ΔP is the skill utility delta, C^+ is the average token cost with specification, and ρ is the token overhead ratio.

Three specifications, TS 29.514 (Policy Authorization), TS 29.507 (Policy Control), and TS 29.512 (Session Management Policy), show positive skill utility deltas ranging from +16.7% to +25.0%. These correspond to bugs where the correct behavior is defined by field optionality or value range constraints in the specification. The remaining seven specifications show zero delta, covering bugs that require only generic defensive programming such as nil guards.

The average token overhead is 12%, reflecting the concise nature of curated 3GPP excerpts compared to full specification documents.

TABLE VI: Specification-as-Skill evaluation (Claude Sonnet 4, multi-turn). Pass$^+$/Pass$^-$: resolve rate with/without spec. ΔP: utility delta. ρ: token overhead. Best viewed in color.

Spec Skill	#Tasks	Pass$^+$	Pass$^-$	ΔP	C^+	ρ
Generic bug type (nil/crash)						
TS 24.501 (NAS)	8	37.5	37.5	0.0	3.4K	+11%
TS 38.413 (NGAP)	5	40.0	40.0	0.0	3.2K	+9%
TS 29.510 (NRF)	4	25.0	25.0	0.0	3.5K	+13%
TS 29.509 (AUSF)	4	25.0	25.0	0.0	3.1K	+8%
TS 29.531 (NSSF)	4	25.0	25.0	0.0	3.3K	+10%
TS 29.518 (AMF)	5	40.0	40.0	0.0	3.6K	+14%
Spec-dependent bug type (validation/logic)						
TS 29.514 (PCF)	6	33.3	16.7	+16.7	3.8K	+15%
TS 29.507 (PCF)	4	25.0	0.0	+25.0	3.7K	+14%
TS 29.512 (SMF)	5	20.0	0.0	+20.0	3.9K	+16%
TS 23.501 (Arch)	5	20.0	20.0	0.0	4.1K	+18%
Overall	50	30.0	24.0	+6.0	3.5K	+12%

These results demonstrate that specification utility is conditional on bug type in the telecommunications domain. 3GPP specifications define the ground truth for correct protocol behavior, making them effective for protocol-semantic bugs, but they provide no value for bugs requiring only generic defensive programming.

VI. Conclusion

We have introduced SWE-Bench 5G, the first benchmark for evaluating AI coding agents on telecom network engineering tasks. The benchmark provides 210 validated instances spanning 9 Network Functions from three open-source 5G core projects, with a dual test strategy, Docker-based reproducibility, and an automated construction pipeline. Our evaluation across four LLMs reveals that agentic iteration is the primary bottleneck, with multi-turn feedback improving resolve rates from 0% to 10 to 30%. The specification-as-skill experiment demonstrates that 3GPP protocol excerpts provide targeted improvement on specification-dependent bugs while offering no benefit for generic defensive checks.

Future work includes expanding the dataset with cross-NF coordination tasks, integrating full agentic tools with native file editing capabilities, and evaluating generalization to additional 5G implementations.

REFERENCES

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[2] free5GC Team, "free5GC: Open source 5G core network," https://github.com/free5gc/free5gc, 2024.

[3] Open5GS Team, "Open5GS: Open source implementation of 5G core and EPC," https://github.com/open5gs/open5gs, 2024.

[4] Linux Foundation, "Magma: Platform for building access networks and modular network services," https://github.com/magma/magma, 2024.

[5] Z. Yao *et al.*, "SWE-bench mobile: An evaluation benchmark for mobile app engineering," *arXiv preprint arXiv:2602.09540*, 2025.

[6] Z. Shi *et al.*, "BeyondSWE: A comprehensive benchmark for evaluating code agents beyond narrow bug fixing," *arXiv preprint arXiv:2603.03194*, 2025.

[7] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, "SWE-skills-bench: Do agent skills actually help in real-world software engineering?" *arXiv preprint arXiv:2603.15401*, 2025.